

JAVA EE 8 FRONT-END - ANGULAR (V19)

Módulo 6: Proyecto Final y Buenas Prácticas

Introducción al Módulo

El desarrollo del proyecto final integrador constituye la cúspide formativa del itinerario de especialización en Angular 19. A lo largo de esta unidad se orquestan y consolidan de manera transversal todas las competencias asimiladas en los módulos precedentes: la arquitectura libre de módulos mediante Componentes Standalone, la reactividad granular de precisión basada en Signals frente a Zone.js, la navegación protegida con Functional Guards, la carga diferida bajo demanda con loadComponent, la captura de parámetros con Component Input Binding y el consumo estructurado de APIs REST tolerantes a fallos con HttpClient y operadores avanzados de RxJS. Este manual técnico establece los estándares de calidad de software requeridos en entornos productivos y guía al alumno paso a paso en la materialización de una consola de control empresarial moderna, robusta y de alto rendimiento.

Estructura Temática y Hoja de Ruta Formativa

Este manual técnico está concebido como una guía teórico-práctica secuencial articulada en los siguientes puntos principales:

- **1 Organización Moderna del Proyecto:** Estructuración por dominios de negocio (Feature-Based Architecture), árbol limpio de directorios y desacoplamiento absoluto de NgModules.
- **2 Análisis de Buenas Prácticas en Angular 19:** Reactividad fina con Signals de complejidad $\mathcal{O}(k)$, control de flujo nativo integrado en el compilador (@if, @for) y prevención de fugas de memoria con takeUntilDestroyed().
- **3 Enunciado del Proyecto Final Integrador: CloudServices Hub:** Contexto industrial de telecomunicaciones, analogía de la torre de control y especificación de requerimientos técnicos obligatorios.
- **4 Implementación Completa del Código Fuente:** Desglose exhaustivo, ordenado y comentado de los ficheros de configuración, seguridad perimetral, servicios de datos y componentes standalone.
- **5 Fases de desarrollo:** Hoja de ruta didáctica guiada paso a paso orientada al alumno para inicializar con CLI, codificar en sus carpetas exactas, verificar y compilar la solución final.

1 Organización Moderna del Proyecto

La ingeniería de software en frontend empresarial contemporáneo exige prescindir por completo de los antiguos archivos organizativos (@NgModule) y estructurar las aplicaciones siguiendo una filosofía orientada a características o dominios funcionales (*Feature-Based Structure*).

Este enfoque garantiza un desacoplamiento estricto, previene colisiones de dependencias, facilita el mantenimiento en equipos grandes y optimiza la división de paquetes de distribución (*Code Splitting*).

1.1 Estructura de Capas Empresariales

El árbol de carpetas de una aplicación robusta en Angular 19 se divide en tres capas fundamentales:

- **Capa Core** (`src/app/core/`): Alberga artefactos globales de infraestructura e instancias singleton que se configuran una única vez y abastecen a toda la plataforma. Incluye servicios de autenticación de sesión, interceptores de cabeceras HTTP y guardianes funcionales de seguridad de rutas.
- **Capa Features** (`src/app/features/`): Contiene los módulos funcionales de negocio de la aplicación (por ejemplo: autenticación, catálogo general, visualización analítica o formularios de alta). Cada característica agrupa sus propios componentes de vista y estilos específicos.
- **Capa Shared** (`src/app/shared/`): Agrupa elementos visuales genéricos y utilidades puras que pueden ser consumidos de manera transversal por cualquier vista de negocio, tales como componentes de tarjetas, indicadores de carga (spinners), badges de estado e interfaces de datos de TypeScript.

A continuación, se detalla la estructura limpia y estandarizada adoptada para la plataforma corporativa **CloudServices Hub**:

```
src/app/
|-- core/                <-- Elementos globales de infraestructura no editables
|   |-- guards/          <-- Guardias funcionales de proteccion de rutas
|   |-- interceptors/    <-- Interceptores HTTP (telemetria, tokens, cabeceras)
|   |-- services/        <-- Servicios globales Singleton (autenticacion, API)
|-- features/            <-- Modulos funcionales de negocio (vistas e interaccion)
|   |-- auth/            <-- Inicio de sesion y control de credenciales
|   |-- dashboard/       <-- Listado de infraestructura y metricas de salud SLA
|   |-- detail/          <-- Vista detallada de nodo con telemetria en tiempo real
|   |-- form/            <-- Formulario reactivo de alta y reglas de validacion
|-- shared/              <-- Artefactos transversales reutilizables
|   |-- components/      <-- Componentes UI de presentacion (tarjetas, badges)
|   |-- models/          <-- Contratos de tipado estricto e interfaces de datos
|-- app.routes.ts         <-- Cartografia de enrutamiento perezoso (loadComponent)
|-- app.config.ts         <-- Proveedores funcionales de configuracion global
|-- app.component.ts      <-- Componente standalone raiz de la aplicacion
```

2 Análisis de Buenas Prácticas en Angular 19

2.1 Reactividad Fina con Signals vs Zone.js

La integración de las señales (*Signals*) constituye el avance más relevante en el modelo de ejecución y rendimiento de Angular. En la arquitectura clásica basada en `Zone.js`, el framework monkey-patchea todas las APIs asíncronas del navegador (temporizadores, eventos de ratón, llamadas AJAX) y, ante cualquier interacción, se ve forzado a recorrer descendientemente la totalidad del árbol jerárquico de componentes para evaluar mutaciones:

$$T_{\text{Zone.js}} = \mathcal{O}(N) \quad (1)$$

Donde N es el total de componentes montados en el documento. En contraste, las primitivas reactivas nativas de Angular 19 (`signal`, `computed`, `linkedSignal`) establecen una gráfica de dependencias directa entre el dato y el nodo exacto del DOM que lo consume, permitiendo una sincronización quirúrgica:

$$T_{\text{re-render}} = \mathcal{O}(k) \quad \text{donde } k \ll N \quad (2)$$

Donde k representa únicamente la cantidad exacta de enlaces del DOM dependientes del valor alterado, eliminando la sobrecarga computacional en aplicaciones complejas.

2.2 Uso de @if y @for para la Optimización del Renderizado

El control de flujo declarativo integrado a nivel de compilador en Angular 19 suprime la sobrecarga sintáctica y de memoria de las directivas estructurales clásicas:

- **Bloque Condicional (@if):** Se prefiere frente a la directiva legada `*ngIf` debido a su compatibilidad nativa con la hidratación asíncrona del SSR y a que no requiere importar `CommonModule`. Además, permite anidar de forma fluida bloques `@else if` y `@else`.
- **Bloque Iterativo (@for):** Exige de forma obligatoria la expresión `track`. El compilador utiliza este identificador unívoco como clave primaria para la reconciliación atómica de elementos en el DOM, descartando funciones auxiliares complejas como `trackBy`.
- **Bloque de Lista Vacía (@empty):** Se integra nativamente dentro del cuerpo de `@for` y se renderiza de forma automática cuando la colección evaluada carece de elementos o es nula.

2.3 Prevención de Fugas de Memoria con takeUntilDestroyed()

Al suscribir flujos asíncronos u operaciones de red con `HttpClient` que no utilicen la conversión directa a señales mediante `toSignal()`, es mandatorio prevenir retenciones involuntarias de memoria (*Memory Leaks*). Para ello, Angular 19 provee el operador funcional `takeUntilDestroyed()`:

```
import { Component, inject, OnInit } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { takeUntilDestroyed } from '@angular/core/rxjs-interop';

@Component({
  selector: 'app-data-consumer',
  standalone: true,
  template: `<p>Consumidor de datos activos en red.</p>`
})
export class DataConsumerComponent implements OnInit {
  private http = inject(HttpClient);

  constructor() {
    // Al invocarse dentro del constructor, deduce de forma automática el Injection Context
    this.http.get('https://api.empresa.org/data')
      .pipe(takeUntilDestroyed())
      .subscribe(res => console.log('Payload sincronizado:', res));
  }
}
```

```
ngOnInit(): void {}  
}
```

Listing 1: p2-practicas/data-consumer.component.ts

3 Enunciado del Proyecto Final Integrador: CloudServices Hub

3.1 Contexto del Problema

Has sido seleccionado como ingeniero principal de frontend para diseñar e implementar la consola de monitorización central de la infraestructura de telecomunicaciones de una organización multinacional distribuida. La plataforma, denominada **CloudServices Hub**, debe permitir a los operadores autenticarse de forma segura, examinar el catálogo actual de microservicios y pasarelas de red mediante llamadas REST, auditar métricas de latencia en tiempo real, inspeccionar la ficha técnica de cada nodo mediante paso dinámico de parámetros y registrar nuevas pasarelas a través de formularios reactivos gobernados por validaciones personalizadas.

3.1.1 La Analogía de la Torre de Control Aeroportuaria

Para asimilar el rol de cada artefacto sin necesidad de dominar previamente el framework, visualiza esta analogía didáctica:

- **El Control Perimetral (AuthGuard):** Es la garita de acceso donde se exige el pase de seguridad. Si el visitante no acredita token activo, es escoltado de vuelta a la recepción (/login).
- **El Radar Central (DashboardComponent):** Muestra en tiempo real todos los vuelos y calcula la latencia media para avisar si el espacio aéreo está saturado (violación de SLA).
- **La Ficha Quirúrgica (DetailComponent):** Al seleccionar un avión por su matrícula (:id), la pantalla sintoniza su sensor particular sin recargar toda la interfaz.
- **El Hangar de Incorporación (FormComponent):** Permite dar de alta una nueva nave verificando que cumpla los protocolos de seguridad antes de autorizar el despegue.

3.2 Requerimientos Técnicos Obligatorios

La solución técnica desarrollada debe cumplir de forma rigurosa con los siguientes cinco pilares de arquitectura:

1. **Autenticación Segura Simulada:** Control de acceso con estado reactivo gobernado por Signals en AuthService, blindando las rutas mediante el guardián funcional authGuard que redirija a /login con un UrlTree.
2. **Carga Perezosa Completa (Lazy Loading):** Matriz de navegación en app.routes.ts articulada mediante loadComponent, generando fragmentos (*chunks*) independientes transferidos bajo demanda.
3. **Mapeo Paramétrico Directo (Component Input Binding):** La ficha técnica del nodo debe capturar el parámetro dinámico :id directamente como una propiedad de entrada gracias a withComponentInputBinding().

4. **Consumo REST y Resiliencia con RxJS:** Centralización de llamadas asíncronas mediante `HttpClient` inyectado funcionalmente, incorporando reintentos con `retry(2)` y manejo de excepciones mediante `catchError`.
5. **Formulario Reactivo con Validación Especial:** Alta de nodos con `FormGroup` y `FormControl`, incorporando una regla personalizada que impida el prefijo "prod-" en nombres asignados a entornos de desarrollo.

4 Implementación Completa del Código Fuente

4.1 Proveedores de Configuración Global: `app.config.ts`

Centraliza los servicios del inyector raíz, habilitando el enrutador reactivo con captura de parámetros y el cliente HTTP funcional:

```
import { ApplicationConfig, provideZoneChangeDetection } from '@angular/core';
import { provideRouter, withComponentInputBinding } from '@angular/router';
import { provideHttpClient } from '@angular/common/http';
import { appRoutes } from './app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection({ eventCoalescing: true }),
    provideRouter(appRoutes, withComponentInputBinding()),
    provideHttpClient()
  ]
};
```

Listing 2: `src/app/app.config.ts`

4.2 Cartografía de Navegación: `app.routes.ts`

Mapeo de rutas asíncronas con división de código protegidas por el guardián perimetral:

```
import { Routes } from '@angular/router';
import { authGuard } from '../core/guards/auth.guard';

export const appRoutes: Routes = [
  { path: '', redirectTo: 'login', pathMatch: 'full' },
  {
    path: 'login',
    loadComponent: () => import('../features/auth/login.component')
      .then(m => m.LoginComponent)
  },
  {
    path: 'dashboard',
    loadComponent: () => import('../features/dashboard/dashboard.component')
      .then(m => m.DashboardComponent),
    canActivate: [authGuard]
  },
  {
    path: 'service/new',
    loadComponent: () => import('../features/form/form.component')
      .then(m => m.FormComponent),
    canActivate: [authGuard]
  },
  {
```

```
path: 'service/:id',
loadComponent: () => import('./features/detail/detail.component')
  .then(m => m.DetailComponent),
canActivate: [AuthGuard]
},
{ path: '**', redirectTo: 'login' }
];
```

Listing 3: src/app/app.routes.ts

4.3 Componente Contenedor Raíz: app.component.ts

Componente standalone base que actúa como marco de la Single Page Application (SPA), inyectando el espacio de renderizado dinámico con RouterOutlet:

```
import { Component } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  template: `
    <main class="app-viewport">
      <router-outlet></router-outlet>
    </main>
  `,
  styles: [`
    .app-viewport { min-height: 100vh; background-color: #F8FBFE; font-family: 'Noto Sans', sans-serif;
    ↵ }
  `]
})
export class AppComponent {}
```

Listing 4: src/app/app.component.ts

4.4 Servicio de Autenticación Reactivo: auth.service.ts

Gestiona el estado de sesión y token en memoria mediante primitivas signal() y computed():

```
import { Injectable, signal, computed } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class AuthService {
  private tokenSignal = signal<string | null>(null);

  public readonly isAuthenticated = computed(() => this.tokenSignal() !== null);
  public readonly activeToken = computed(() => this.tokenSignal());

  login(username: string, clave: string): boolean {
    if (username.trim() && clave === 'admin19') {
      const fakeToken = `JWT_TOKEN_HUB_${Math.random().toString(36).substring(2)}`;
      this.tokenSignal.set(fakeToken);
      return true;
    }
    return false;
  }
}
```

```
}

logout(): void {
  this.tokenSignal.set(null);
}
}
```

Listing 5: src/app/core/services/auth.service.ts

4.5 Guardián Funcional de Seguridad: auth.guard.ts

Función pura que valida la autorización activa e interrumpe tránsitos no válidos redirigiendo mediante UrlTree:

```
import { inject } from '@angular/core';
import { CanActivateFn, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

export const authGuard: CanActivateFn = (route, state) => {
  const authService = inject(AuthService);
  const router = inject(Router);

  if (authService.isAuthenticated()) {
    return true;
  }

  return router.createUrlTree(['/login']);
};
```

Listing 6: src/app/core/guards/auth.guard.ts

4.6 Servicio de Telemetría y Consumo REST: telemetry.service.ts

Capa de abstracción de datos para el repositorio de microservicios con tolerancia activa a contingencias de red:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient, HttpResponseError } from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { catchError, retry } from 'rxjs/operators';

export interface Microservicio {
  id: string;
  nombre: string;
  ip: string;
  entorno: 'desarrollo' | 'produccion';
  estado: 'saludable' | 'critico';
  latencia: number;
}

@Injectable({
  providedIn: 'root'
})
export class TelemetryService {
  private http = inject(HttpClient);
  private apiUrl = 'https://jsonplaceholder.typicode.com/todos';

  private localDb: Microservicio[] = [
```

```

    { id: '1', nombre: 'API Gateway Principal', ip: '10.0.0.1', entorno: 'produccion', estado: '
      ↳ saludable', latencia: 24 },
    { id: '2', nombre: 'Autenticacion IAM', ip: '10.0.0.2', entorno: 'produccion', estado: 'saludable',
      ↳ latencia: 12 },
    { id: '3', nombre: 'Microservicio Facturacion', ip: '10.0.1.5', entorno: 'desarrollo', estado: '
      ↳ critico', latencia: 185 },
    { id: '4', nombre: 'CDN Balanceador Norte', ip: '10.0.5.1', entorno: 'produccion', estado: '
      ↳ saludable', latencia: 8 }
  ];

  getServices(): Microservicio[] {
    return [...this.localDb];
  }

  getServiceById(id: string): Microservicio | undefined {
    return this.localDb.find(m => m.id === id);
  }

  registerService(nuevo: Microservicio): void {
    this.localDb.push(nuevo);
  }

  fetchRemoteData(): Observable<any> {
    return this.http.get<any>(this.apiUrl).pipe(
      retry(2),
      catchError(this.handleError)
    );
  }

  private handleError(error: HttpErrorResponse) {
    let userMsg = 'Error de comunicacion con el cluster central.';
    if (error.status === 404) {
      userMsg = 'El recurso de telemetria solicitado no existe.';
    }
    return throwError(() => new Error(userMsg));
  }
}

```

Listing 7: src/app/core/services/telemetry.service.ts

4.7 Componente de Acceso: login.component.ts

Controlador standalone de inicio de sesión con alertas de error reactivas:

```

import { Component, inject, signal } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { Router } from '@angular/router';
import { AuthService } from '../../core/services/auth.service';

@Component({
  selector: 'app-login',
  standalone: true,
  imports: [CommonModule, FormsModule],
  template: `
    <div class="login-box">
      <h2>ORGANIZACION TI - CLOUD SERVICES HUB</h2>
      <p class="subtitle">IT Formacion - Angular v19</p>
      <div class="card">

```

```

    <h3>Iniciar Sesión</h3>
    @if (errorMsg()) {
      <div class="alert danger">{{ errorMsg() }}</div>
    }
    <div class="field">
      <label>Usuario Técnico:</label>
      <input [(ngModel)]="usuario" placeholder="Ej: admin" class="form-control" />
    </div>
    <div class="field">
      <label>Contraseña Corporativa:</label>
      <input type="password" [(ngModel)]="clave" placeholder="admin19" class="form-control" />
    </div>
    <button (click)="ejecutarLogin()" class="btn btn-primary">Acceder al Sistema</button>
  </div>
</div>
`,
styles: [
  .login-box { max-width: 440px; margin: 60px auto; font-family: 'Noto Sans', sans-serif; }
  h2 { color: #1F2758; text-align: center; font-size: 19px; }
  .subtitle { text-align: center; color: #64748B; font-size: 13px; margin-top: -8px; }
  .card { background: white; padding: 24px; border-radius: 8px; border: 1px solid #CBD5E1; }
  .field { margin-bottom: 14px; }
  label { font-size: 12px; font-weight: bold; color: #2D3748; display: block; margin-bottom: 4px; }
  .form-control { width: 100%; padding: 8px; border: 1px solid #CBD5E1; border-radius: 4px; box-sizing
    ↪ : border-box; }
  .btn { width: 100%; padding: 10px; background: #1E75B8; color: white; border: none; border-radius: 4
    ↪ px; font-weight: bold; cursor: pointer; }
  .alert { background: #FEF2F2; border-left: 4px solid #DC2626; color: #991B1B; padding: 10px; font-
    ↪ size: 12px; margin-bottom: 14px; }
]
})
export class LoginComponent {
  usuario = '';
  clave = '';
  errorMsg = signal<string | null>(null);

  private authService = inject(AuthService);
  private router = inject(Router);

  ejecutarLogin(): void {
    this.errorMsg.set(null);
    const exito = this.authService.login(this.usuario, this.clave);
    if (exito) {
      this.router.navigate(['/dashboard']);
    } else {
      this.errorMsg.set('Credenciales invalidas. Ingrese usuario y clave corporativa (admin19).');
    }
  }
}
}

```

Listing 8: src/app/features/auth/login.component.ts

4.8 Componente de Control Central: dashboard.component.ts

Panel principal con cálculo computado de latencias promedio y directiva declarativa @for:

```

import { Component, inject, signal, computed, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { Router, RouterLink } from '@angular/router';

```

```

import { TelemetryService, Microservicio } from '../../core/services/telemetry.service';
import { AuthService } from '../../core/services/auth.service';

@Component({
  selector: 'app-dashboard',
  standalone: true,
  imports: [CommonModule, RouterLink],
  template: `
    <div class="dashboard-wrap">
      <header class="main-head">
        <div>
          <h1>Servidor Central de Telemetria</h1>
          <p class="user-info">Sesion activa: <span class="badge">Operador de Infraestructura</span></p>
        </div>
        <button (click)="salir()" class="btn-logout">Desconectar</button>
      </header>

      <section class="overview">
        <div class="metric-card bg-primary">
          <h4>Latencia Media Global</h4>
          <h2>{{ latenciaMedia() }} ms</h2>
        </div>
        <div class="metric-card" [class.stable]="sistemaEstable()" [class.warning]="!sistemaEstable()">
          <h4>Diagnostico de SLA</h4>
          <h2>{{ estadoSistema() }}</h2>
        </div>
      </section>

      <div class="actions-bar">
        <h3>Lista de Control de Nodos</h3>
        <button routerLink="/service/new" class="btn-add">+ Registrar Microservicio</button>
      </div>

      <div class="table-container">
        <table class="data-table">
          <thead>
            <tr>
              <th>ID</th>
              <th>Nombre de Servicio</th>
              <th>Direccion IP</th>
              <th>Entorno</th>
              <th>Latencia</th>
              <th>Estado</th>
              <th>Accion</th>
            </tr>
          </thead>
          <tbody>
            @for (ms of listaMicroservicios(); track ms.id) {
              <tr>
                <td>#{{ ms.id }}</td>
                <td class="bold">{{ ms.nombre }}</td>
                <td class="font-mono">{{ ms.ip }}</td>
                <td><span class="env-tag" [class]="ms.entorno">{{ ms.entorno | uppercase }}</span></td>
                <td><strong [class.lag]="ms.latencia > 100">{{ ms.latencia }} ms</strong></td>
                <td>
                  <span class="dot" [class.green]="ms.estado === 'saludable'" [class.red]="ms.estado ===
↪ 'critico'"></span>
                  {{ ms.estado | uppercase }}
                </td>
              </tr>
            }
          </tbody>
        </table>
      </div>
    </div>
  `
})

```

```

        <td>
            <button (click)="navegarDetalle(ms.id)" class="btn-view">Ver Detalles</button>
        </td>
    </tr>
    } @empty {
    <tr>
        <td colspan="7" class="empty-state">No se registran nodos de infraestructura activos.</td>
    </tr>
    }
</tbody>
</table>
</div>
</div>
`,
styles: [
    .dashboard-wrap { max-width: 1000px; margin: 30px auto; font-family: 'Noto Sans', sans-serif; }
    .main-head { display: flex; justify-content: space-between; align-items: center; border-bottom: 2px
    ↪ solid #1E75B8; padding-bottom: 12px; margin-bottom: 24px; }
    h1 { color: #1F2758; margin: 0; font-size: 22px; }
    .badge { background: #1F2758; color: white; padding: 2px 8px; border-radius: 4px; font-size: 11px; }
    .btn-logout { background: #DC2626; color: white; border: none; padding: 6px 14px; border-radius: 4px
    ↪ ; cursor: pointer; }
    .overview { display: grid; grid-template-columns: 1fr 1fr; gap: 20px; margin-bottom: 24px; }
    .metric-card { background: white; padding: 18px; border-radius: 8px; border: 1px solid #E2E8F0; text
    ↪ -align: center; }
    .metric-card.bg-primary { background: #1E75B8; color: white; border: none; }
    .metric-card h4 { margin: 0; font-size: 11px; text-transform: uppercase; }
    .metric-card h2 { margin: 8px 0 0 0; font-size: 26px; }
    .stable { background: #F0FDF4; color: #059669; border-color: #86EFAC; }
    .warning { background: #FEF2F2; color: #DC2626; border-color: #FCA5A5; }
    .actions-bar { display: flex; justify-content: space-between; align-items: center; margin-bottom: 12
    ↪ px; }
    .btn-add { background: #1F2758; color: white; border: none; padding: 8px 14px; border-radius: 4px;
    ↪ cursor: pointer; font-weight: bold; }
    .table-container { background: white; border-radius: 8px; border: 1px solid #CBD5E1; overflow:
    ↪ hidden; }
    .data-table { width: 100%; border-collapse: collapse; }
    .data-table th, .data-table td { padding: 10px 12px; text-align: left; border-bottom: 1px solid #
    ↪ F1F5F9; font-size: 13px; }
    .data-table th { background: #F8FBFE; color: #1F2758; font-weight: bold; }
    .env-tag { padding: 2px 6px; border-radius: 3px; font-size: 10px; font-weight: bold; }
    .env-tag.produccion { background: #EFF6FF; color: #1E75B8; }
    .env-tag.desarrollo { background: #FEF3C7; color: #D97706; }
    .dot { width: 8px; height: 8px; border-radius: 50%; display: inline-block; margin-right: 4px; }
    .dot.green { background: #059669; }
    .dot.red { background: #DC2626; }
    .lag { color: #DC2626; }
    .btn-view { background: #F1F5F9; border: 1px solid #CBD5E1; padding: 4px 10px; border-radius: 4px;
    ↪ cursor: pointer; }
    .empty-state { text-align: center; color: #64748B; padding: 20px; font-style: italic; }
]
})
export class DashboardComponent implements OnInit {
    private telemetryService = inject(TelemetryService);
    private authService = inject(AuthService);
    private router = inject(Router);

    listaMicroservicios = signal<Microservicio[]>([]);

```

```

latenciaMedia = computed(() => {
  const list = this.listaMicroservicios();
  if (list.length === 0) return 0;
  const sum = list.reduce((acc, curr) => acc + curr.latencia, 0);
  return Math.round(sum / list.length);
});

sistemaEstable = computed(() => this.latenciaMedia() < 80);
estadoSistema = computed(() => this.sistemaEstable() ? 'Optimo (SLA Garantizado)' : 'Alerta de Retardo
  ↳ ');

ngOnInit(): void {
  this.listaMicroservicios.set(this.telemetryService.getServices());
}

navegarDetalle(id: string): void {
  this.router.navigate(['/service', id]);
}

salir(): void {
  this.authService.logout();
  this.router.navigate(['/login']);
}
}

```

Listing 9: src/app/features/dashboard/dashboard.component.ts

4.9 Componente de Registro Reactivo: form.component.ts

Formulario reactivo con validador condicional cruzado a nivel de grupo:

```

import { Component, inject, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormGroup, FormControl, Validators, ReactiveFormsModule, AbstractControl, ValidationErrors }
  ↳ from '@angular/forms';
import { Router, RouterLink } from '@angular/router';
import { TelemetryService, Microservicio } from '../../../core/services/telemetry.service';

@Component({
  selector: 'app-form',
  standalone: true,
  imports: [CommonModule, ReactiveFormsModule, RouterLink],
  template: `
    <div class="form-wrapper">
      <h2>Alta de Nueva Pasarela de Red</h2>
      <p class="desc">Asegura los parametros de conectividad antes de incorporarlos al cluster.</p>
      <form [formGroup]="registroForm" (ngSubmit)="guardar()">
        <div class="card">
          <div class="form-group">
            <label>Nombre del Microservicio:</label>
            <input formControlName="nombre" class="form-control" placeholder="Ej. Servicio Pasarela" />
            @if (registroForm.get('nombre')?.touched && registroForm.get('nombre')?.invalid) {
              <div class="err-text">El nombre es mandatorio y exige al menos 4 caracteres.</div>
            }
          </div>

          <div class="form-group">
            <label>Direccion IP (Subred 10.0.x.x):</label>

```

```

        <input formControlName="ip" class="form-control" placeholder="10.0.1.10" />
        @if (registroForm.get('ip')?.touched && registroForm.get('ip')?.invalid) {
            <div class="err-text">La IP debe cumplir el formato de subred privada corporativa (10.0.x.
↪ x).</div>
        }
    </div>

    <div class="grid">
        <div class="form-group">
            <label>Entorno:</label>
            <select formControlName="entorno" class="form-control">
                <option value="desarrollo">Desarrollo (Sandbox)</option>
                <option value="produccion">Produccion (Live)</option>
            </select>
        </div>
        <div class="form-group">
            <label>Latencia Estimada (ms):</label>
            <input type="number" formControlName="latencia" class="form-control" />
        </div>
    </div>

    @if (registroForm.errors?.['prefijoReservado']) {
        <div class="alert danger">
            <strong>Regla de Seguridad:</strong> No se admiten nombres con prefijo "prod-" en entornos
↪ de desarrollo.
        </div>
    }

    <div class="btn-group">
        <button type="submit" [disabled]="registroForm.invalid" class="btn btn-save">Confirmar
↪ Registro</button>
        <button type="button" routerLink="/dashboard" class="btn btn-cancel">Cancelar</button>
    </div>
</form>
</div>
`,
styles: [`
.form-wrapper { max-width: 600px; margin: 30px auto; font-family: 'Noto Sans', sans-serif; }
h2 { color: #1F2758; margin: 0; }
.desc { color: #64748B; font-size: 13px; margin: 6px 0 20px 0; }
.card { background: white; padding: 24px; border-radius: 8px; border: 1px solid #CBD5E1; }
.form-group { margin-bottom: 16px; }
label { font-size: 12px; font-weight: bold; color: #2D3748; display: block; margin-bottom: 4px; }
.form-control { width: 100%; padding: 8px; border: 1px solid #CBD5E1; border-radius: 4px; box-sizing
↪ : border-box; }
.grid { display: grid; grid-template-columns: 1fr 1fr; gap: 15px; }
.err-text { color: #DC2626; font-size: 11px; margin-top: 4px; }
.btn-group { display: flex; gap: 10px; margin-top: 20px; }
.btn { padding: 9px 16px; border: none; border-radius: 4px; font-weight: bold; cursor: pointer; }
.btn-save { background: #1E75B8; color: white; }
.btn-save:disabled { background: #CBD5E1; cursor: not-allowed; }
.btn-cancel { background: #F1F5F9; color: #475569; border: 1px solid #CBD5E1; }
.alert { background: #FEF2F2; border-left: 4px solid #DC2626; color: #991B1B; padding: 10px; font-
↪ size: 12px; margin-top: 14px; }
`]
})
export class FormComponent implements OnInit {
    private telemetryService = inject(TelemetryService);

```

```

private router = inject(Router);

registroForm!: FormGroup;

ngOnInit(): void {
  this.registroForm = new FormGroup({
    nombre: new FormControl('', [Validators.required, Validators.minLength(4)]),
    ip: new FormControl('', [Validators.required, Validators.pattern(/^10\.0\.\d{1,3}\.\d{1,3}$/)]),
    entorno: new FormControl('desarrollo', Validators.required),
    latencia: new FormControl(10, [Validators.required, Validators.min(1)])
  }, {
    validators: [this.validadorPrefijoEntorno]
  });
}

validadorPrefijoEntorno(control: AbstractControl): ValidationErrors | null {
  const nombre = control.get('nombre')?.value as string;
  const entorno = control.get('entorno')?.value as string;

  if (nombre && entorno === 'desarrollo' && nombre.toLowerCase().startsWith('prod-')) {
    return { prefijoReservado: true };
  }
  return null;
}

guardar(): void {
  if (this.registroForm.invalid) return;
  const val = this.registroForm.value;
  const nuevo: Microservicio = {
    id: (this.telemetryService.getServices().length + 1).toString(),
    nombre: val.nombre,
    ip: val.ip,
    entorno: val.entorno,
    estado: val.latencia > 100 ? 'critico' : 'saludable',
    latencia: val.latencia
  };
  this.telemetryService.registerService(nuevo);
  this.router.navigate(['/dashboard']);
}
}

```

Listing 10: src/app/features/form/form.component.ts

4.10 Componente de Detalle Técnico: detail.component.ts

Captura el parámetro de URL mediante `@Input()` `id` e integra la primitiva reactiva `linkedSignal`:

```

import { Component, Input, OnInit, inject, signal, linkedSignal } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterLink } from '@angular/router';
import { TelemetryService, Microservicio } from '../../../core/services/telemetry.service';

@Component({
  selector: 'app-detail',
  standalone: true,
  imports: [CommonModule, RouterLink],
  template: `
    <div class="detail-wrap">

```

```

<h2>Telemetria Quirurgica del Nodo</h2>
<p class="desc">Parametros de red recuperados mediante Component Input Binding.</p>

@if (microservicio()) {
  <div class="card">
    <header class="card-head">
      <h3>{{ microservicio()?.nombre }}</h3>
      <span class="badge" [class]="microservicio()?.estado">{{ microservicio()?.estado | uppercase
↪ }}</span>
    </header>

    <table class="detail-table">
      <tr>
        <td><strong>Identificador:</strong></td>
        <td class="font-mono">UUID-NODO-{{ microservicio()?.id }}</td>
      </tr>
      <tr>
        <td><strong>Direccion IP:</strong></td>
        <td class="font-mono">{{ microservicio()?.ip }}</td>
      </tr>
      <tr>
        <td><strong>Entorno Tecnologico:</strong></td>
        <td>{{ microservicio()?.entorno | uppercase }}</td>
      </tr>
      <tr>
        <td><strong>Latencia de Respuesta:</strong></td>
        <td><strong>{{ microservicio()?.latencia }} ms</strong></td>
      </tr>
      <tr>
        <td><strong>Asignacion de Memoria (linkedSignal):</strong></td>
        <td>{{ limiteConsumo() }}</td>
      </tr>
    </table>

    <div class="sandbox">
      <p class="text-xs"><strong>Simulacion Reactiva:</strong> Modifica la latencia para comprobar
↪ la respuesta de linkedSignal.</p>
      <div class="flex gap-2">
        <button (click)="degradarNodo()" class="btn btn-warn">Forzar Sobrecarga (+50ms)</button>
        <button (click)="restaurarNodo()" class="btn btn-ok">Estabilizar Nodo (10ms)</button>
      </div>
    </div>

    <button routerLink="/dashboard" class="btn-back">Regresar al Catalogo</button>
  </div>
} @else {
  <div class="alert danger">
    El microservicio con identificador <strong>#{{ id }}</strong> no se encuentra en el cluster
↪ activo.
  </div>
  <button routerLink="/dashboard" class="btn-back">Regresar al Catalogo</button>
}
</div>
`,
styles: [
  .detail-wrap { max-width: 600px; margin: 30px auto; font-family: 'Noto Sans', sans-serif; }
  h2 { color: #1F2758; margin: 0; }
  .desc { color: #64748B; font-size: 13px; margin: 6px 0 20px 0; }
  .card { background: white; padding: 24px; border-radius: 8px; border: 1px solid #CBD5E1; }

```

```

    .card-head { display: flex; justify-content: space-between; align-items: center; border-bottom: 1px
    ↪ solid #F1F5F9; padding-bottom: 10px; margin-bottom: 16px; }
    .card-head h3 { margin: 0; color: #1F2758; }
    .badge { padding: 3px 8px; border-radius: 4px; font-size: 11px; font-weight: bold; color: white; }
    .badge.saludable { background: #059669; }
    .badge.critico { background: #DC2626; }
    .detail-table { width: 100%; margin-bottom: 20px; }
    .detail-table td { padding: 8px 0; border-bottom: 1px dashed #E2E8F0; font-size: 13px; }
    .font-mono { font-family: monospace; }
    .sandbox { background: #FFFBEF; border: 1px solid #FDE68A; padding: 12px; border-radius: 6px; margin
    ↪ -bottom: 16px; }
    .text-xs { font-size: 12px; color: #92400E; margin-top: 0; }
    .flex { display: flex; gap: 8px; }
    .btn { padding: 6px 12px; border: none; border-radius: 4px; cursor: pointer; font-size: 12px; font-
    ↪ weight: bold; }
    .btn-warn { background: #D97706; color: white; }
    .btn-ok { background: #059669; color: white; }
    .btn-back { background: #1F2758; color: white; border: none; padding: 8px 14px; border-radius: 4px;
    ↪ cursor: pointer; text-decoration: none; font-size: 12px; }
    .alert { background: #FEF2F2; border-left: 4px solid #DC2626; color: #991B1B; padding: 12px; font-
    ↪ size: 13px; margin-bottom: 14px; }
  `]
})
export class DetailComponent implements OnInit {
  @Input() id!: string;

  private telemetryService = inject(TelemetryService);
  microservicio = signal<Microservicio | undefined>(undefined);

  limiteConsumo = linkedSignal({
    source: this.microservicio,
    computation: (servicio) => {
      if (!servicio) return 'No determinado';
      return servicio.entorno === 'produccion' ? 'Ilimitado (Alta Demanda)' : 'Restringido (512MB RAM)';
    }
  });

  ngOnInit(): void {
    const ms = this.telemetryService.getServiceById(this.id);
    this.microservicio.set(ms);
  }

  degradarNodo(): void {
    const act = this.microservicio();
    if (act) {
      this.microservicio.set({ ...act, latencia: act.latencia + 50, estado: 'critico' });
      this.limiteConsumo.set('Estrangulado a 128MB RAM (Medida de Contingencia)');
    }
  }

  restaurarNodo(): void {
    const act = this.microservicio();
    if (act) {
      this.microservicio.set({ ...act, latencia: 10, estado: 'saludable' });
    }
  }
}

```

Listing 11: src/app/features/detail/detail.component.ts

5 Fases de desarrollo

Esta sección proporciona una guía metodológica y secuencial orientada al alumno para ejecutar, estructurar, verificar y desplegar de forma autónoma el proyecto **CloudServices Hub**. Cada fase incluye los comandos exactos de terminal de la CLI de Angular, las rutas donde se crean los artefactos y la explicación de su encaje arquitectónico:

Fase 1: Inicialización del Entorno y Configuración Global

- **Aprovisionamiento con Angular CLI:** Abre tu terminal y ejecuta la orden de creación del proyecto. Configuramos preprocesador SCSS, generamos el sistema de enrutamiento integrado y activamos el modo estricto de TypeScript:

```
ng new cloud-services-hub --style scss --routing --strict
```

- **Ubicación de los archivos creados:** El comando genera la raíz del workspace en `cloud-services-hub/`. Accede al directorio de trabajo con `cd cloud-services-hub`.
- **Configuración de Proveedores Globales (`src/app/app.config.ts`):** Abre este archivo existente y registra `provideZoneChangeDetection({ eventCoalescing: true })`, `provideRouter(appRoutes, withComponentInputBinding())` y `provideHttpClient()`. Esto habilita la comunicación HTTP en toda la aplicación y permite que las URLs pasen parámetros dinámicos directamente a los componentes sin configurar suscripciones.
- **Marco Maestro SPA (`src/app/app.component.ts`):** Asegúrate de que el componente raíz importe `RouterOutlet` y que su plantilla contenga la directiva `<router-outlet></router-outlet>`, actuando como el contenedor visual donde se proyectarán las distintas pantallas.

Fase 2: Modelado y Servicios de Infraestructura (Capa Core)

- **Creación de la Capa de Servicios:** Para centralizar la lógica de negocio y las comunicaciones asíncronas bajo el patrón Singleton, ejecuta los siguientes generadores esquemáticos:

```
ng g s core/services/auth
ng g s core/services/telemetry
```

- **Ubicación física resultante:** La CLI creará automáticamente los ficheros en `src/app/core/services/auth.service.ts` y `src/app/core/services/telemetry.service.ts`.
- **Implementación de Tipos y Telemetría:** Dentro de `telemetry.service.ts`, define la interfaz `Microservicio` con sus contratos de tipado estricto (`id`, `nombre`, `ip`, `entorno`, `latencia`, `estado`). Implementa el acceso al array en memoria y la llamada a `this.http.get()` combinando los operadores de RxJS `retry(2)` y `catchError()`.
- **Implementación de Autenticación Reactiva:** En `auth.service.ts`, declara la señal mutable privada `tokenSignal = signal<string | null>(null)` y expón la señal computada pública de solo lectura `isAuthenticated = computed(() => this.tokenSignal() !== null)`.

Fase 3: Seguridad Perimetral y Cartografía de Rutas

- **Generación del Guardián Funcional:** Genera la función de seguridad perimetral de rutas ejecutando:

```
ng g g core/guards/auth --type canActivate
```

- **Ubicación física:** El archivo se emplaza en `src/app/core/guards/auth.guard.ts`.
- **Lógica de Protección:** Edita la función `authGuard` inyectando `AuthService` y `Router` mediante la función moderna `inject()`. Si `isAuthenticated()` es falso, retorna inmediatamente `router.createUrlTree(['/login'])` para abortar el tránsito no autorizado.
- **Matriz de Rutas Perezosas (`src/app/app.routes.ts`):** Configura las entradas de navegación asociando `loadComponent: () => import('./ruta').then(m => m.Component)` a cada ruta. Asocia `canActivate: [authGuard]` a las vistas protegidas: `dashboard`, `service/new` y `service/:id`.

Fase 4: Construcción de Vistas de Acceso y Panel Principal

- **Scaffolding de Componentes de Acceso y Dashboard:** Genera los componentes standalone de negocio mediante las siguientes órdenes:

```
ng g c features/auth/login --standalone --inline-style --inline-template
ng g c features/dashboard/dashboard --standalone --inline-style --inline-template
```

- **Ubicación de los archivos:** Se generarán en `src/app/features/auth/login.component.ts` y `src/app/features/dashboard/dashboard.component.ts`.
- **Lógica de Login:** Integra las directivas `CommonModule` y `FormsModule` para enlazar los campos de texto con `[(ngModel)]`. Al validar las credenciales (`admin19`), navega programáticamente con `this.router.navigate(['/dashboard'])`.
- **Lógica del Dashboard Central:** Inyecta `TelemetryService` y almacena los microservicios en una señal `listaMicroservicios`. Calcula la latencia media agregada con `computed()` y utiliza el bloque nativo `@for (ms of listaMicroservicios(); track ms.id)` en la plantilla HTML para renderizar la tabla con alto rendimiento.

Fase 5: Captura Avanzada y Mapeo Paramétrico de Detalle

- **Scaffolding de Formulario y Ficha Técnica:** Genera los dos componentes interactivos restantes de la capa de características:

```
ng g c features/form/form --standalone --inline-style --inline-template
ng g c features/detail/detail --standalone --inline-style --inline-template
```

- **Ubicación resultante:** Se ubicarán en `src/app/features/form/form.component.ts` y `src/app/features/detail/detail.component.ts`.
- **Formulario Reactivo con Validación Cruzada:** En `form.component.ts`, importa `ReactiveFormsModule`, instancia un `FormGroup` con sus `FormControl` y añade la función validadora a nivel grupal `validadorPrefijoEntorno` para impedir nombres que comiencen por "prod-" si el entorno es desarrollo.
- **Mapeo Directo y linkedSignal en Detalle:** En `detail.component.ts`, declara la propiedad de entrada `@Input() id!: string`. El enrutador asignará este valor de la URL automáticamente gracias a `withComponentInputBinding()`. Declara la primitiva reactiva `linkedSignal` para gestionar la cuota de memoria sincronizada con el entorno tecnológico del nodo consultado.

Fase 6: Verificación de Calidad, Pruebas y Compilación

- **Arranque del Servidor Local:** Inicia la compilación en caliente para verificar el proyecto:

```
ng serve --open
```

- **Validación del Bloqueo Perimetral:** Con la aplicación abierta en el navegador, intenta escribir directamente en la barra de direcciones `http://localhost:4200/dashboard`. Verifica que el guardián te expulsa de forma instantánea hacia la pantalla de `/login`.
- **Verificación de Chunks en Red:** Abre las Herramientas de Desarrollador (F12) en la pestaña *Network/Red*. Inicia sesión con `admin` y `admin19`, y observa cómo el archivo JavaScript correspondiente al Dashboard se transfiere por la red únicamente en el instante en que se solicita su acceso (*Lazy Loading real*).
- **Compilación para Producción:** Genera el paquete final optimizado y verifica la integridad del tipado estático:

```
ng build --configuration production
```

La carpeta de distribución `dist/cloud-services-hub/` contendrá los paquetes finales minificados, listos para su despliegue en cualquier servidor web empresarial.